

CPP NOTES – DAY 31

“This” Operator

In C++, the this operator is a special pointer that refers to the object for which a member function is called. It is implicitly passed to all non-static member functions and can be used to access the object's members and methods.

Key Principles

1. Implicit Pointer: The this pointer is automatically passed to non-static member functions. It is not available in static member functions because they can be called without any object.
2. Type of this: For a class X, the type of this is X*. If a member function is declared as const, the type of this is const X*.
3. R-value: In modern C++, the this pointer is an r-value, meaning it cannot be assigned to another pointer.

```
#include<iostream>
using namespace std;

class Test {
private:
    int x;
public:
    void setX(int x) {
        this->x = x; // 'this->x' refers to the member variable
    }
    void print() {
        cout << "x = " << x << endl;
    }
};

int main() {
    Test obj;
    obj.setX(20);
    obj.print(); // Output: x = 20
    return 0;
}
```

Pointer to Class

A pointer that holds the **address of an object** (i.e., instance) of a class.

```
class MyClass {
public:
    void sayHello() {
        std::cout << "Hello\n";
    }
};

int main() {
    MyClass obj;
    MyClass* ptr = &obj; // Pointer to class
    ptr->sayHello();      // Access via pointer
    return 0;
}
```

Pointer to Class Member

A pointer that refers to a **specific member** (data or function) **inside** a class, not to the object itself.

```
class MyClass {
public:
    int value;
};

int main() {
    MyClass obj;
    obj.value = 42;

    int MyClass::*ptrToData = &MyClass::value; // Pointer to member variable
    std::cout << obj.*ptrToData << std::endl;   // Accessing member via object
    return 0;
}
```

INHERITANCE

Inheritance is a feature of **Object-Oriented Programming** that allows a class (**derived class**) to acquire properties and behaviors (data members and member functions) of another class (**base class**). It promotes **code reusability, extensibility, and polymorphism**.

There are several types of inheritance, depending on how classes are related:

1. Single Inheritance

A subclass inherits from one superclass.

2. Multiple Inheritance

A subclass inherits from more than one superclass.

3. Multilevel Inheritance

A class is derived from a child class, which is itself derived from another class.

4. Hierarchical Inheritance

Multiple subclasses inherit from the same superclass.

5. Hybrid Inheritance

A combination of two or more types of inheritance. This can lead to the **diamond problem**, especially in multiple inheritance scenarios.

FYI

- *To return reference to the calling object, this can be used.*
- *Diff between pointer to a class and pointer to a member of class*
- *All inheritance are "is- a relationship" - An "is-a" relationship represents inheritance. It means a derived class is a specialized type of a base class. If Dog is a type of Animal, then Dog is-a Animal.*
- *A "has-a" relationship represents composition or aggregation. It means one class contains another as a member.*

If Car contains an Engine, then Car has-a Engine.

- *Composition, Aggregation and association are made up of has-a relationship*
- *Message passing in oops- method to method communication(method from one class calls method from another class to communicate)*